Parker Hannifin — Current State Web Architecture (Observed).

Reverse-engineered from public, client-side evidence only (cookies, response headers, bundle paths, config APIs). No internal documentation or credentials were used. Last observed: 2026-10-10.

Yuji Nakagawa, Expert AEM Solutoin Consultant



Hybrid modern front-end over legacy commerce back-end.

www.parker.com is AEM on-prem running the official Adobe SPA Editor framework at Level 3 (full WYSIWYG integration) — not a server-rendered HTL shell with isolated React "islands," and not a fully decoupled/headless SPA either.

ph.parker.com and **help.parker.com**, by contrast, are fully decoupled Next.js apps (Turbopack bundles) with no visible AEM SPA Editor SDK — consistent with Adobe's "Full SPA / headless" pattern rather than Level 3.

Identity has been modernized to Okta (federating to Azure AD), decoupling login UX from the legacy commerce engine. **Commerce/session system of record** is still IBM WebSphere Commerce — storeId, langId, catalogId, and WCTrustedToken all persist as the backbone of account state.

Edge, identity & commerce signals

Level 3
AEM SPA Editor WYSIWYG integration on www.parker.com.

23551
WCS storeId surfaced in HFStatic JSON on api.parker.com.

98×
MapTo() component bindings in clientlib-react.min.js bundle.

2-tier
Akamai + Azure App Gateway edge (bot/WAF then LB affinity).

Front-end applications in the Parker Hannifin stack

www.parker.com	ph.parker.com	help.parker.com	Separate platforms
AEM on-prem SPA Editor Level 3 React clientlib + .model.json	**Next.js (Turbopack)** Product catalog Where-to-Buy	**Next.js (Turbopack)** Help & Support HR-Benefits (/wtb_support)	**External sites** blog, crossref, parkerstore, investors

Backed by Akamai, Azure, Okta & WCS

Three supporting layers wrap the Parker Hannifin core stack.

Edge / CDN

Akamai CDN + WAF + Bot Manager (AkamaiGHost, bm_sv/bm_so cookies), fronted by Azure Application Gateway for affinity.

Identity / SSO

Okta on login.parker.com, SAML SSO federated to Azure AD via Okta app parkerexternal_parkercomazure_1.

Legacy commerce

IBM WebSphere Commerce (storeId 23551) remains the identity & session of record, fronted by the api.parker.com BFF layer.

Component Notes & Evidence.

AEM + SPA

www.parker.com frontend

AEM on-prem shell + React clientlib boots client-side.

6+ tools

Third-party MarTech layer

GA4, Clarity, Eloqua, Qualtrics, Dynatrace, mPulse.

Edge — Akamai

CDN, WAF, Bot Manager — blocks non-browser/bot requests (403 on direct curl).

server: AkamaiGHost header; bm_sv / bm_so / bm_sc / bm_lso / bm_mi cookies (Akamai Bot Manager); akamai-grn header.

Edge — Azure Application Gateway

Session affinity / load balancing in front of app tier.

ApplicationGatewayAffinity / ApplicationGatewayAffinityCORS cookies.

Component evidence across layers.

Layer	Component	Role	Evidence
Edge	**Akamai mPulse (Boomerang/BOOMR)**	Real User Monitoring	`c.go-mpulse.net/api/config.json request, BOOMR_API_key global`
Identity	**Okta**	SAML SSO, custom domain login.parker.com	`URL /app/{app-label}/{okta-instance-id}/sso/saml; app label references Azure (parkerexternal_parkercomazure_1)`
Frontend	**www.parker.com**	AEM on-prem, Hybrid / SPA Editor pattern — Level 3 (full WYSIWYG integration). Server returns a near-empty shell; React app boots client-side, fetches .model.json, and maps AEM resource types to React components via the official SPA Editor SDK.	`Raw response: <h1 id="h1tagheader">Home</h1><div id="spa-root"></div> + /etc.clientlibs/parker-com/clientlibs/clientlib-react.min.*.js. Bundle contains PageModelManager, ModelClient, MapTo (98×), withMappable. templateName: parker-homepage-template.`
Frontend	**ph.parker.com**	Product catalog, "Where to Buy" — Next.js app.	`/_next/static/chunks/*.js, Turbopack bundle names`
Frontend	**help.parker.com**	Help & Support, HR/Benefits — Next.js app under /wtb_support.	`/wtb_support/_next/static/chunks/*.js`
Frontend	**blog.parker.com, crossref.parker.com, parkerstore.com, investors.parker.com**	Separate subdomains/platforms, not yet fingerprinted in this session.	Linked from main nav/footer only.
API	**api.parker.com**	Gateway/BFF layer feeding frontends.	`GET /prod/HFStatic/?key=divisionId:687PDC~locale:en (header/footer nav JSON incl. storeId); /prod/chatwebservice/chatDetail`
Legacy backend	**IBM WebSphere Commerce (WCS)**	Underlying commerce/session/identity-of-record engine.	`storeId: "23551" in HFStatic JSON; langId=-1 cookie; WCTrustedToken var names (prod_/dev_/test_ prefixed) read via getCookie() and fed into analytics as user_id`
MarTech	**GA4 / Google Ads, Microsoft Clarity, Oracle Eloqua, Qualtrics, Dynatrace**	Analytics, lead-gen, VoC, APM.	`Script URLs + cookies: _ga*, _clck / _clsk, ELOQUA, siteintercept.qualtrics.com, dtCookie / dtPC / rxVisitor`

Open questions (not yet confirmed from client-side alone).

- Whether `api.parker.com` sits in front of WCS directly or via an additional BFF/integration layer.
- Hosting location/provider for WCS itself (Azure, on-prem, or another cloud).
- Whether `blog.parker.com`, `crossref.parker.com`, `parkerstore.com`, and `investors.parker.com` share any backend with the core stack or are fully independent (e.g. investors.parker.com is very likely a third-party IR SaaS platform).
- Whether the AEM on-prem instance behind `www.parker.com` is a single shared publish farm serving all AEM-authored pages, or whether other sub-paths/templates still render fully server-side HTL without the SPA Editor shell (only the homepage template was directly confirmed in this session).
- Whether page-level AEM Core Components (non-SPA templates, if any exist) coexist with the Level 3 SPA templates on the same instance — i.e. is this a partial/phased SPA Editor rollout, or is every `www.parker.com` template built this way.